

Custom Memory Allocator

malloc / free / realloc from scratch in C

Author Samir Mansour

GitHub github.com/Samir204

Project Summer Portfolio – Systems Programming

Date July 2026

Language C / C++

Contents

1	Introduction	3
1.1	What This Project Implements	3
1.2	Repository Structure	3
2	Background and Motivation	5
2.1	How the Heap Works	5
2.2	Why Not Just Call sbrk Every Time	5
2.3	Fragmentation	6
3	Data Structures	7
3.1	The Block Header	7
3.2	Magic Numbers	7
3.3	The Heap Linked List	8
4	Allocation Strategies	9
4.1	First-Fit	9
4.2	Best-Fit	9
4.3	Block Splitting	10
4.4	Coalescing	10
4.5	The Buddy System	11
5	Core Implementation	13
5.1	my_malloc	13
5.2	Alignment	14
5.3	my_free with Safety Checks	14
5.4	my_realloc with In-Place Growth	15
6	Thread Safety	16
6.1	Trade-offs of a Single Global Lock	16
7	Benchmarking	17
7.1	Methodology	17
7.2	Results	18

7.3	Analysis	18
7.3.1	First-Fit	18
7.3.2	Best-Fit	18
7.3.3	Buddy System	18
8	Testing	20
9	Known Limitations and Future Work	22
9.1	Current Limitations	22
9.2	Future Work	22
10	Conclusions	23

1. Introduction

When you write `malloc(32)` in a C program, the standard library does a surprising amount of work behind the scenes. It finds a free chunk of heap memory, tracks it, hands you a pointer, and later takes it back when you call `free()`. This project builds all of that machinery from scratch in C, without using any of the standard library allocator functions.

The goal is not to beat glibc in a benchmark. The goal is to understand exactly what glibc is doing, why it makes the trade-offs it does, and to be able to talk about memory management at a level most programmers never reach. That depth is what separates someone who *knows C* from someone who actually understands how computers work underneath.

This report documents every part of the implementation: the data structures, all three allocation strategies (first-fit, best-fit, and buddy system), thread safety, corruption detection, and the benchmarking results that show the trade-offs between each approach.

1.1 What This Project Implements

- A complete heap allocator using `sbrk` and `mmap` system calls
- Three allocation strategies: first-fit, best-fit, and buddy system
- Block splitting to avoid wasting memory inside large free blocks
- Block coalescing to merge adjacent free blocks and reduce fragmentation
- Thread safety using a POSIX mutex
- Double-free and invalid pointer detection using magic number headers
- In-place `realloc` growth when the next block is free and adjacent
- A benchmarking harness comparing throughput and fragmentation ratio across all three strategies

1.2 Repository Structure

```
allocator/  
  allocator.h      public API and struct definitions  
  allocator.c      main implementation (first-fit, best-fit)
```

buddy.c	buddy system allocator
test.c	unit test suite
benchmark.c	benchmarking harness
results.csv	benchmark output data
Makefile	build configuration

2. Background and Motivation

Memory management is one of the most fundamental problems in systems programming. Every non-trivial program needs to allocate memory at runtime, and how well it does so directly affects performance, reliability, and security.

2.1 How the Heap Works

When a process starts, the operating system gives it a virtual address space. The **heap** is a region of this space that can grow upward as the program needs more memory. The boundary between the in-use heap and the unmapped region above it is called the **program break**.

There are two main ways a process can ask the OS for more heap memory:

- **sbrk(n)**: moves the program break forward by **n** bytes, returning the old break address. Simple and sequential, but limited to one contiguous region that can only grow at the end.
- **mmap**: maps a new anonymous region of memory anywhere in the address space. More flexible, works in page-sized chunks (4096 bytes each), and individual mappings can be returned to the OS independently.

This allocator uses **sbrk** for the main heap (first-fit and best-fit strategies) and a fixed pool for the buddy system. This mirrors how real allocators like glibc work: **sbrk** for small allocations, **mmap** for large ones.

2.2 Why Not Just Call sbrk Every Time

You cannot call **sbrk** for every individual **malloc**. A program that does a million small allocations would make a million system calls, which would be catastrophically slow. The allocator's job is to call **sbrk** infrequently to grab large chunks of memory, then manage the subdivision and reuse of those chunks in userspace. That management is what makes a memory allocator non-trivial to write correctly.

2.3 Fragmentation

Fragmentation is the core problem that makes allocator design hard. It comes in two forms:

- **External fragmentation:** free memory exists but is split into pieces too small to satisfy a request, even though the total free space would be enough in aggregate.
- **Internal fragmentation:** a block is larger than the requested size, so some bytes inside it are wasted and unavailable to anyone else.

Every allocation strategy makes a different trade-off between these two forms of fragmentation, and between fragmentation and throughput. Measuring that trade-off is the main deliverable of Week 4.

3. Data Structures

3.1 The Block Header

Every chunk of memory managed by the allocator has a small header placed immediately before the user data. The allocator uses this header to track each block without any external bookkeeping:

```
1 typedef struct Block {
2     size_t    size;        // usable bytes after this header
3     int       is_free;     // 1 = available, 0 = in use
4     uint32_t  magic;       // corruption and double-free detection
5     struct Block *next;    // next block in linked list
6     struct Block *prev;    // previous block
7 } Block;
```

On a 64-bit system this struct is 32 bytes. The layout in memory looks like:

```
[ header 32B ][ user data ][ header 32B ][ user data ] ...
```

Given a user pointer `ptr`, recovering the header is always:

```
1 Block *header = (Block*)ptr - 1;
```

And returning the user pointer from a header is always:

```
1 void *user_ptr = (void*)(header + 1);
```

This single pattern is used everywhere in the allocator. Getting it wrong by even one byte silently corrupts memory in ways that are nearly impossible to debug after the fact.

3.2 Magic Numbers

The `magic` field stores a constant that identifies a valid block header. Two values are used:

```
1 #define MAGIC_ALLOC 0xDEADBEEF // block is currently allocated
2 #define MAGIC_FREE  0xDEADDEAD // block has been freed
```


Every new block gets `MAGIC_ALLOC` stamped on creation, and again whenever it is reused by `my_malloc`. When `my_free` is called, it checks the magic field before touching anything else. If it sees `MAGIC_FREE`, the block was already freed – the call is rejected with an error message. If it sees anything other than `MAGIC_ALLOC`, the pointer is invalid and also rejected. Neither case crashes the program, which makes debugging far easier.

A critical bug discovered during development: when `my_malloc` reuses a freed block, the magic field must be reset to `MAGIC_ALLOC`. Missing this caused every subsequent `free` on that block to be reported as a double-free, which produced hundreds of false error messages during the thread safety test.

3.3 The Heap Linked List

All blocks on the heap are linked in a doubly linked list starting at `heap_start`. The list is ordered by address, since blocks are always appended to the end when `extend_heap` is called. Walking this list is how `find_free` and `find_best_fit` search for available blocks.

The linked list approach is simple and correct but $O(n)$ for allocation. Production allocators use segregated free lists (one list per size class) to achieve $O(1)$ or $O(\log n)$ allocation. This is the main area where glibc outperforms this implementation on large heaps.

4. Allocation Strategies

4.1 First-Fit

First-fit scans the block list from the start and returns the first free block large enough to satisfy the request:

```
1 static Block *find_free(size_t size) {
2     Block *cur = heap_start;
3     while (cur) {
4         if (cur->is_free && cur->size >= size)
5             return cur;
6         cur = cur->next;
7     }
8     return NULL;
9 }
```

First-fit is fast because it stops at the first match. The downside is that it tends to fill the beginning of the heap with small leftover fragments, causing subsequent searches to slow down over time as they must skip over many small used blocks before finding a suitable free one.

4.2 Best-Fit

Best-fit scans the entire list and returns the smallest block that is still large enough for the request:

```
1 static Block *find_best_fit(size_t size) {
2     Block *cur = heap_start;
3     Block *best = NULL;
4     while (cur) {
5         if (cur->is_free && cur->size >= size) {
6             if (best == NULL || cur->size < best->size)
7                 best = cur;
8         }
9         cur = cur->next;
10    }
```

```
10     }
11     return best;
12 }
```

Best-fit reduces internal fragmentation because it finds the tightest match, leaving larger free blocks available for future requests. The cost is that it always does a full list scan, making it consistently slower than first-fit. Interestingly, best-fit can actually produce *more* external fragmentation than first-fit on certain workloads, because choosing the tightest match tends to leave many tiny unusable remainders.

4.3 Block Splitting

Both first-fit and best-fit use block splitting. When a free block is significantly larger than needed, it is split into two: an exact-fit allocated block and a smaller free remainder. Without splitting, a 1024-byte free block used to satisfy an 8-byte request would waste 1016 bytes permanently.

```
1  #define MIN_SPLIT (HEADER_SIZE + 8)
2
3  static void split_block(Block *block, size_t size) {
4      if (block->size < size + MIN_SPLIT) return;
5
6      Block *new_block = (Block*)((char*)(block + 1) + size);
7      new_block->size    = block->size - size - HEADER_SIZE;
8      new_block->is_free = 1;
9      new_block->magic    = MAGIC_ALLOC;
10     new_block->next     = block->next;
11     new_block->prev     = block;
12
13     if (block->next) block->next->prev = new_block;
14     block->next = new_block;
15     block->size = size;
16 }
```

`MIN_SPLIT` is set to `HEADER_SIZE + 8`. A block is only split if the remainder would be at least large enough to hold a header plus 8 bytes of usable data. Creating a block smaller than this would just add overhead without any practical benefit.

4.4 Coalescing

When a block is freed, if its neighbours in memory are also free, they are merged into one larger block. This is **coalescing**, and it is essential for preventing the heap from filling

up with unusable tiny fragments over time:

```

1 void coalesce(Block *block) {
2     // merge with next block if it is free
3     if (block->next && block->next->is_free) {
4         block->size += HEADER_SIZE + block->next->size;
5         block->next = block->next->next;
6         if (block->next) block->next->prev = block;
7     }
8     // merge with previous block if it is free
9     if (block->prev && block->prev->is_free) {
10        block->prev->size += HEADER_SIZE + block->size;
11        block->prev->next = block->next;
12        if (block->next) block->next->prev = block->prev;
13    }
14 }

```

Coalescing is called at the end of every `my_free`. The key constraint is that it must only do pointer surgery on existing blocks. Calling `my_malloc` or `my_free` inside `coalesce` would cause infinite recursion.

4.5 The Buddy System

The buddy system is a fundamentally different approach. Instead of blocks being arbitrary sizes, every block is always a power-of-two size. When memory is needed, a large block is halved repeatedly until the smallest power-of-two that fits the request is reached. When freed, a block checks whether its partner (the **buddy**) is also free. If so, they merge back together. Then that merged block checks its buddy, and so on up the chain.

The key insight is how to find a block's buddy. Given a block at address `A` with size `S` and pool base address `base`:

```
buddy_address = base + ((A - base) XOR S)
```

XOR with the block size flips exactly the one bit that separates the block from its buddy. This makes buddy-finding $O(1)$ with no searching required:

```

1 static BuddyBlock *get_buddy(BuddyBlock *block) {
2     size_t offset = (char*)block - buddy_pool;
3     size_t buddy_offset = offset ^ block->size;
4     if (buddy_offset >= BUDDY_POOL_SIZE) return NULL;
5     return (BuddyBlock*)(buddy_pool + buddy_offset);
6 }

```

All allocation and deallocation operations in the buddy system are $O(\log n)$ where n is the number of size levels. The trade-off is internal fragmentation: a 33-byte request gets a 64-byte block, wasting 31 bytes. For workloads with random sizes this can be significant.

5. Core Implementation

5.1 my_malloc

The main allocation function checks the active strategy, searches for a suitable free block (splitting it if found), or extends the heap if nothing fits:

```
1 void *my_malloc(size_t size) {
2     pthread_mutex_lock(&alloc_lock);
3     if (size == 0) {
4         pthread_mutex_unlock(&alloc_lock);
5         return NULL;
6     }
7     size = ALIGN_SIZE(size);
8
9     if (current_strategy == STRATEGY_BUDDY) {
10        pthread_mutex_unlock(&alloc_lock);
11        return buddy_malloc(size);
12    }
13
14    Block *block = (current_strategy == STRATEGY_BEST_FIT)
15                  ? find_best_fit(size)
16                  : find_free(size);
17
18    if (block) {
19        split_block(block, size);
20        block->is_free = 0;
21        block->magic = MAGIC_ALLOC;
22    } else {
23        block = extend_heap(size);
24        if (!block) {
25            pthread_mutex_unlock(&alloc_lock);
26            return NULL;
27        }
28    }
29    pthread_mutex_unlock(&alloc_lock);
```

```
30     return (void*)(block + 1);
31 }
```

5.2 Alignment

Every requested size is rounded up to a multiple of 8 before any operation. This ensures all returned pointers are 8-byte aligned, which is required for correct operation on 64-bit systems. Misaligned pointers cause performance penalties on x86-64 and outright hardware faults on ARM:

```
1 #define ALIGN 8
2 #define ALIGN_SIZE(size) (((size) + (ALIGN - 1)) & ~(ALIGN - 1))
```

5.3 my_free with Safety Checks

my_free validates the pointer using the magic field before touching anything else, then marks the block free and triggers coalescing:

```
1 void my_free(void *ptr) {
2     pthread_mutex_lock(&alloc_lock);
3     if (ptr == NULL) {
4         pthread_mutex_unlock(&alloc_lock);
5         return;
6     }
7     Block *block = (Block*)ptr - 1;
8
9     if (block->magic == MAGIC_FREE) {
10         fprintf(stderr, "ERROR: double-free at %p\n", ptr);
11         pthread_mutex_unlock(&alloc_lock);
12         return;
13     }
14     if (block->magic != MAGIC_ALLOC) {
15         fprintf(stderr, "ERROR: invalid pointer at %p\n", ptr);
16         pthread_mutex_unlock(&alloc_lock);
17         return;
18     }
19     block->magic = MAGIC_FREE;
20     block->is_free = 1;
21     coalesce(block);
22     pthread_mutex_unlock(&alloc_lock);
23 }
```

5.4 my_realloc with In-Place Growth

The upgraded `realloc` first tries to grow the block in place by absorbing the next block if it is free and adjacent. This avoids a costly allocation and copy for the common case where a program is incrementally growing a buffer:

```
1 void *my_realloc(void *ptr, size_t size) {
2     if (!ptr) return my_malloc(size);
3     if (size == 0) { my_free(ptr); return NULL; }
4
5     size = ALIGN_SIZE(size);
6     Block *block = (Block*)ptr - 1;
7
8     if (block->size >= size) return ptr;
9
10    // try to grow in place
11    if (block->next && block->next->is_free) {
12        size_t combined =
13            block->size + HEADER_SIZE + block->next->size;
14        if (combined >= size) {
15            block->size = combined;
16            block->next = block->next->next;
17            if (block->next) block->next->prev = block;
18            return ptr;
19        }
20    }
21
22    // fallback: allocate new, copy, free old
23    void *new_ptr = my_malloc(size);
24    if (!new_ptr) return NULL;
25    memcpy(new_ptr, ptr, block->size);
26    my_free(ptr);
27    return new_ptr;
28 }
```


6. Thread Safety

Real programs call `malloc` and `free` from multiple threads simultaneously. Without protection, two threads could corrupt the block linked list by modifying it at the same time. This allocator uses a single global mutex to prevent that:

```
1 | static pthread_mutex_t alloc_lock = PTHREAD_MUTEX_INITIALIZER;
```

Every public function locks the mutex at entry and unlocks it at every exit point, including all error paths. The most important rule is that a locked function must never call another public function that also tries to acquire the same lock. A single thread cannot acquire the same non-recursive mutex twice without deadlocking.

This is why `my_realloc` unlocks before calling `my_malloc` and `my_free` on its fallback path. Getting this wrong produces a deadlock that is silent under light load and only shows up under concurrent stress testing.

6.1 Trade-offs of a Single Global Lock

A single global lock is the simplest correct approach, but it is the main performance bottleneck under high concurrency. Every thread that wants to allocate memory must wait for every other thread to finish, which completely serialises all allocation operations.

Production allocators solve this with **per-thread arenas**. Each thread gets its own private heap region and only needs the global lock when acquiring a new arena. This is how `jemalloc` and `tcmalloc` achieve near-linear scalability under concurrent workloads.

7. Benchmarking

7.1 Methodology

The benchmarking harness in `benchmark.c` measures each strategy under three allocation patterns designed to simulate different real-world workloads. Each pattern runs with $N = 10,000$ operations.

- **Bulk:** allocate N blocks, then free all of them. Simulates a program that builds a large data structure and tears it down in one go.
- **Interleaved:** allocate half the blocks, then alternately free and reallocate. Simulates a server handling many short-lived requests.
- **Random sizes:** allocate blocks of random sizes between 8 and 512 bytes, then free in reverse order. Simulates real-world mixed allocation patterns and stresses coalescing.

Two metrics are captured per run:

- **Throughput:** operations per second, counting both alloc and free.
- **Fragmentation ratio:** defined as $1 - \frac{\text{largest free block}}{\text{total free memory}}$. A value of 0.0 means all free memory is in one contiguous block. A value of 0.34 means roughly a third of free memory is split into pieces too small to be useful.

7.2 Results

Strategy	Pattern	Throughput (ops/s)	Fragmentation
First-fit	Bulk	~1,600,000	0.00
First-fit	Interleaved	~1,100,000	0.12
First-fit	Random sizes	~800,000	0.34
Best-fit	Bulk	~1,050,000	0.00
Best-fit	Interleaved	~890,000	0.08
Best-fit	Random sizes	~700,000	0.19
Buddy	Bulk	~2,200,000	0.00
Buddy	Interleaved	~1,800,000	0.05
Buddy	Random sizes	~1,400,000	0.28

7.3 Analysis

7.3.1 First-Fit

First-fit is the fastest of the linked-list strategies because it stops searching as soon as it finds a match. The 0.34 fragmentation ratio on random-size workloads is the cost: roughly a third of free memory becomes split into pieces too small to reuse, and the allocator has to call `sbrk` again instead of recycling existing space. Over a long-running program, this gets progressively worse.

7.3.2 Best-Fit

Best-fit reduces fragmentation on interleaved workloads (0.08 versus 0.12 for first-fit) by finding tighter matches and leaving larger contiguous free blocks intact. The cost is a consistent throughput penalty because it always scans the entire list. The random-size fragmentation (0.19) is better than first-fit but still significant, because choosing the tightest fit tends to leave many tiny remainders that are too small to satisfy most future requests.

7.3.3 Buddy System

The buddy system is the fastest across all three patterns because finding and splitting power-of-two blocks is $O(\log n)$ with no linear list scanning. Bulk allocation produces zero fragmentation because all same-size blocks split and coalesce cleanly. The random-size fragmentation (0.28) is entirely due to internal fragmentation from size rounding:

a request for 33 bytes gets a 64-byte block, wasting 31 bytes. This is a fundamental characteristic of the buddy system, not a correctness bug.

8. Testing

The test suite in `test.c` covers the full public API and all safety features. Every test uses `assert()` so a failure stops the program immediately with a clear message rather than silently producing wrong output.

Test	What it checks
<code>test_basic_alloc</code>	<code>malloc</code> returns non-null, written data survives, <code>free</code> does not crash
<code>test_alignment</code>	every pointer is 8-byte aligned regardless of requested size
<code>test_zero_alloc</code>	<code>malloc(0)</code> returns NULL
<code>test_calloc_zeroed</code>	<code>calloc</code> returns properly zeroed memory
<code>test_realloc_grows</code>	<code>realloc</code> to a larger size preserves all existing data
<code>test_double_free</code>	freeing a pointer twice prints an error and does not crash
<code>test_realloc_inplace</code>	<code>realloc</code> absorbs the next free block and returns the same pointer
<code>test_thread_safety</code>	two threads doing 1000 <code>alloc/free</code> cycles each produce no corruption

To build and run the full test suite:

```
gcc -Wall -Wextra -pthread -o test allocator.c buddy.c test.c &&
./test
```

Expected output:

```
PASS test_basic_alloc
PASS test_alignment
PASS test_zero_alloc
```

```
PASS test_calloc_zeroed
PASS test_realloc_grows
ERROR: double-free at 0x...    <- intentional, from
      test_double_free
PASS test_double_free
PASS test_realloc_inplace
PASS test_thread_safety

All tests passed.
```

The single ERROR line is intentional. `test_double_free` deliberately frees the same pointer twice to verify that the detection logic catches and reports it without crashing the program.

9. Known Limitations and Future Work

9.1 Current Limitations

- **Single global mutex:** all allocation operations are serialised. Under high thread contention this becomes the dominant bottleneck. Per-thread arenas (the design used by jemalloc and tcmalloc) would fix this.
- **No large-allocation path:** everything goes through `sbrk`. Real allocators switch to `mmap` for requests above a threshold (typically 128KB) so those chunks can be returned directly to the OS on `free` rather than sitting in the heap forever.
- **O(n) allocation for first-fit and best-fit:** both strategies scan the full linked list. As the heap grows, allocation time grows with it. Segregated free lists (one list per size class) would reduce this to O(1) for the common case.
- **Fixed-size buddy pool:** the buddy allocator is backed by a static 64KB array. A real implementation would grow the pool on demand using `mmap`.
- **No heap validation tool:** beyond the magic number checks in `my_free`, there is no way to walk the entire heap and check for consistency (overlapping blocks, corrupted next/prev pointers, etc.).

9.2 Future Work

- **Per-thread caches:** give each thread a small private pool of recently freed blocks to serve from without acquiring the global lock. This is the most impactful improvement for concurrent workloads.
- **Size-class segregation:** maintain separate free lists for common sizes (8, 16, 32, 64, 128 bytes) to make typical allocations O(1) regardless of heap size.
- **mmap fallback for large allocations:** use `mmap` for requests above 64KB and `munmap` on `free`, so large allocations do not fragment the main heap permanently.
- **Heap consistency checker:** a debug-mode function that walks every block, validates magic numbers, checks that linked list pointers are consistent, and reports anomalies with addresses and sizes.
- **Allocation statistics:** record total allocated bytes, peak usage, number of `sbrk` calls, and allocation counts per size class for profiling and tuning.

10. Conclusions

Building a memory allocator from scratch is one of the most effective ways to genuinely understand what happens below the C standard library. After implementing this project, calling `malloc` no longer feels like magic. It is a linked list walk, a pointer bump, a mutex lock, and roughly 400 lines of careful C.

The benchmarking results confirm what the theory predicts. First-fit is simple and fast but fragments badly under realistic workloads. Best-fit trades speed for slightly better memory efficiency but is not the clear winner the textbook description implies. The buddy system wins on throughput by avoiding linear scanning, but pays for it with internal fragmentation from size rounding. None of these is the right answer for all workloads, which is exactly why production allocators like `jemalloc` combine multiple strategies and size classes depending on the allocation size and thread activity.

The most important lesson from this project is that correctness in memory management is extremely unforgiving. A single missed pointer update in `coalesce`, a forgotten magic field reset in `malloc`, or a mutex unlock in the wrong place in `realloc` produces bugs that pass simple tests and only surface under concurrent or long-running workloads. Writing the test suite and the benchmarking harness was as valuable as writing the allocator itself – they are what made those bugs visible.